

Processo

Un processo è un programma in esecuzione.

Esso fornisce un'*astrazione* fondamentale del SO, consentendogli di semplificare l'allocazione delle risorse. Ogni processo ha un *proprio flusso di controllo*, e tutti i processi progrediscono ma solo uno è attivo in un dato momento.

Gerarchie di processi

Il SO crea solo un processo di `init`, mentre i sottoprocessi sono creati in modo *indipendente*.

Creazione e terminazione di un processo

Un processo viene creato a seguito di uno di questi quattro eventi principali:

1. *Inizializzazione* del sistema;
2. Esecuzione di una *chiamata di sistema per la creazione* di un processo da parte di un processo in esecuzione (`fork()`);
3. *Richiesta dell'utente* di creare un nuovo processo (es. tramite `bash`);
4. *Avvio di un lavoro* in modalità batch.

Invece, un processo viene terminato a seguito di quattro condizioni tipiche:

1. Uscita normale (volontaria);
 2. Uscita a causa di un errore (volontaria);
 3. Errore 'fatale' (involontario);
 4. Ucciso da un altro processo (involontario).
- `fork` : Crea un *nuovo* processo
 - Il figlio è un 'clone' del genitore, con cui condivide alcune risorse (es. file aperti, segmento di codice, variabili d'ambiente e directory corrente ereditate)
 - `exec` : Esegue un nuovo processo (utilizzato in combinazione con `fork`)
 - `exit` : Termina il processo volontariamente ('stato di uscita' comunicato al processo genitore)

- **kill** : Segnale ad un processo (o gruppo di processi) che può causare la terminazione involontaria di un processo.

Un processo può trovarsi in uno di questi tre stati:

1. **Running** - In esecuzione (utilizzando la CPU);
2. **Ready** - Pronto (eseguibile, temporaneamente fermo per consentire l'esecuzione di un altro processo);
3. **Blocked** - Bloccato (finché non si verifica un evento esterno).

Nella tabella dei processi del SO sono memorizzate, per ogni processo, le informazioni:

- ID (PID), Utente (UID), Gruppo (GUID);
- Spazio degli indirizzi di memoria
- Registri HW (Es. Program Counter);
- File aperti;
- Segnali;
- Interrupt;

Interrupt e Signal

Meccanismi utilizzati nei SO e nelle applicazioni per gestire eventi asincroni

Interrupt:

Evento di origine HW, gestito tramite la *Interrupt Service Routine*, viene utilizzato per la comunicazione tra HW e SW, con una risposta pronta agli eventi HW. Si verifica in modo asincrono ma gestito immediatamente.

Signal:

Evento di origine HW, generati da un processo o dal SO, come gestore di segnali personalizzati o comportamenti predefiniti. Viene utilizzato per la gestione di condizioni eccezionali nelle applicazioni, e viene inviato asincronicamente ma può essere gestito in modo sincrono.

Per deallocare la CPU a favore dello scheduler ci si affida alla ISR, che *permette allo scheduler di ottenere periodicamente il controllo*. Si utilizza un *interrupt vector* associato a ciascun dispositivo I/O e linea di interrupt e contenente l'indirizzo iniziale di una procedura interna fornita dal SO.

Al verificarsi di un interrupt:

1. L'HW impila il Program Counter e le altre info del processo;

2. L'HW carica il nuovo PC dal vettore degli interrupt;
3. La procedura in linguaggio Assembly salva i registri ed imposta un nuovo stack;
4. Il servizio di interrupt C viene eseguito;
5. Lo scheduler decide quale processo deve essere eseguito successivamente;
6. La procedura C ritorna al codice Assembly
7. La procedura Assembly avvia il nuovo processo

Ogni volta che si verifica un interrupt lo scheduler ottiene il controllo ed agisce come mediatore (Un processo non può cedere CPU ad un altro processo senza passare attraverso lo scheduler).

Tipi di segnali:

- HW-indotti (es. `SIGKILL`);
- SW-indotti (es. `SIGQUIT` o `SIGPIPE`).

Azioni possibili:

- `Term, Ign, Core, Stop, Cont` ;
- Azione predefinita per ogni segnale, tipicamente sovrascrivibile;
- I segnali possono essere tipicamente bloccati e le azioni ritardate.

Gestione dei segnali:

- Il processo registra il gestore del segnale;
- Il SO invia il segnale e consente al processo di eseguire l'handler;
- Il contesto di esecuzione corrente deve essere salvato/ripristinato.

Thread

Per thread si intende mini processi leggeri associati a dei processi che permettono la multiesecuzione in parallelo efficienti in termine di spazio e tempo, e con una comunicazione ed una sincronizzazione *semplici*.

I thread **risiedono nello stesso spazio degli indirizzi di un singolo processo**, e tutti gli scambi di informazioni avvengono tramite dati condivisi tra i thread. Ogni thread ha il proprio *stack*, i propri *registri HW* ed il proprio *stato*. Ciascun thread può chiamare *qualsiasi* chiamata di sistema supportata dal SO *per conto del processo* a cui appartiene

Thread call	Description
<code>pthread_create</code>	Crea un nuovo thread
<code>pthread_exit</code>	Termina il thread chiamante
<code>pthread_join</code>	Attende "l'uscita" di uno specifico thread

Thread call	Description
<code>pthread_yield</code>	Rilascia la CPU per consentire l'esecuzione di un altro thread
<code>pthread_attr_init</code>	Crea ed inizializza la struttura di attributi di un thread
<code>pthread_attr_destroy</code>	Rimuove la struttura di attributi di un thread

Sincronizzazione dei processi

I processi hanno bisogno di un modo per *comunicare* (condividere dati) e quindi di un modo per *sincronizzarsi* (per tenere conto delle dipendenze e per evitare che si intralcino a vicenda).

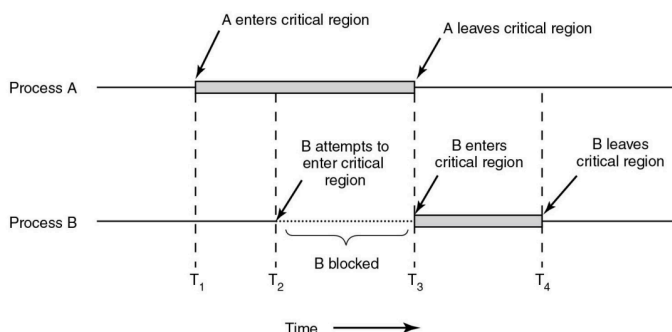
Si introduce il concetto di *race conditions*, ovvero il fenomeno in cui più processi accedono alla stessa risorsa condivisa. La lettura/scrittura di un file deve essere un'operazione *atomica*, altrimenti i processi possono "gareggiare" tra loro e giungere a conclusioni errate.

Regione critica

Requisiti per evitare le race conditions:

1. Due processi non possono trovarsi contemporaneamente all'interno delle proprie regioni critiche;
2. Non si possono fare ipotesi su velocità/numero di CPU;
3. Nessun processo in esecuzione (al di fuori della propria regione critica) può bloccare altri processi;
4. Nessun processo deve aspettare tempo infinito per entrare nella propria regione critica.

Mutua esclusione



Esclusione reciproca tramite regioni critiche.

Mutua esclusione con Busy Waiting: i

processi o thread attendono in un ciclo attivo (ossia senza far nulla di produttivo) di accedere alla regione critica. Lo fa attraverso:

- **Disabilitazione degli Interrupt:** Un processo che si trova in regione critica blocca gli interrupt della CPU finché non ha finito. Funziona solo con i sistemi Unicore
 - **Blocco delle variabili:** 0 se il blocco è libero, 1 se occupato.
 - **Alternanza stretta:** Variabile `turn` per tenere traccia del turno in cui un processo può entrare nella sua regione critica.
- NON SOLUZIONE:** Non permette ai processi di entrare nelle loro regioni critiche per due volte di seguito, ma soprattutto *un processo fuori regione critica può effettivamente bloccarne un altro.*

Algoritmo di Peterson

Alice e Bob vogliono usare un'unica postazione computer in un ufficio. Ma ci sono delle regole:

1. Solo una persona può usare il computer alla volta.
2. Se entrambi vogliono usarlo contemporaneamente, devono decidere chi va per primo.

Idea dell'algoritmo

- Alice o Bob devono segnalare il loro interesse a usare il computer.
- Se l'altro non è interessato, la persona interessata può usarlo subito.
- Se entrambi mostrano interesse, registrano il loro nome su un foglio. Ma se scrivono quasi allo stesso tempo, l'ultimo nome sul foglio ha la precedenza.
- La persona che non ha la precedenza aspetta finché l'altra ha finito.
- Una volta finito, la persona che ha usato il computer segnala che ha finito, e l'altra può iniziare.

L'effettiva **soluzione** è *lasciare che un processo in attesa di entrare nella sua regione critica restituisca volontariamente la CPU allo scheduler.* - `SLEEP` e `WAKEUP`.

- `sleep()` - Imposta il suo stato su `blocked` e lascia CPU allo scheduler;
- `wakeup()` - Imposta il suo stato su `ready` e lascia CPU allo scheduler.

Mutua esclusione - Semafori

Utilizzati per contare e gestire i `wakeup`.

Valori: può essere 0 (nessun wakeup) o $n > 0$ (num. di wakeup in attesa).

Operazioni:

- `down` :
 - Se il valore del semaforo è > 0 allora viene decrementato, ed il processo continua la sua esecuzione.

- Se il valore del semaforo è 0, il processo che ha invocato `down` viene bloccato e messo in una coda di attesa associata al semaforo (in pratica va "a dormire").
- `up` :
 - Se il valore del semaforo è 0 vuol dire che ci sono processi nella coda di attesa, che vengono "svegliati";
 - In ogni caso, il valore viene incrementato ed il processo continua la sua esecuzione.

Le operazioni dei semafori sono *atomiche*, perciò indivisibili, per evitare conflitti.

Mutex e pthreads

Mutex - Versione esplicita e semplificata dei semafori, usata per gestire la mutua esclusione di risorse o codice condiviso, quando *non bisogna contare* accessi o altri fenomeni. Due stati:

- `locked` - bloccato;
- `unlocked` - sbloccato

E due procedure principali:

- `mutex_lock` ;
- `mutex_unlock` ;

Quando un thread vuole accedere ad una regione critica, chiama `mutex_lock` : se il mutex è sbloccato allora il thread può entrare, altrimenti attende. Al termine dell'accesso, il thread chiama `mutex_unlock` per liberare la risorsa.

Non c'è busy waiting, se un thread non può acquisire un lock, chiama `thread_yield` per cedere la CPU ad un altro thread.

Chiamata	Descrizione
<code>pthread_mutex_init</code>	Inizializza un mutex
<code>pthread_mutex_destroy</code>	Distrukge un mutex liberando le risorse associato. Solo se il mutex non è detenuto da alcun thread
<code>pthread_mutex_lock</code>	Blocca un mutex, sospendendo l'operazione del thread chiamante se il mutex è già occupato.
<code>pthread_mutex_trylock</code>	Tenta di bloccare un mutex senza sospendere l'esecuzione.
<code>pthread_mutex_unlock</code>	Sblocca un mutex, permettendo ad altri thread di acquisirlo. Chiamato solo dal thread che detiene il lock

- I semafori possono essere utilizzati sia per la gestione dell'accesso alle risorse condivise che per la sincronizzazione tra thread. Tuttavia, non avendo una semantica di proprietà (qualsiasi thread può incrementarne o decrementarne il valore indipendentemente da chi lo ha modificato l'ultima volta), è preferibile utilizzarli per la sincronizzazione tra thread.
- I mutex vengono principalmente utilizzati per la mutua esclusione poiché a differenza dei semafori, avendo la semantica di proprietà, garantiscono che due thread diversi non possono accedere alla stessa regione critica nello stesso istante.

Mutua esclusione - Monitor

Concetto di sincronizzazione ad alto livello per semplificare la scrittura dei programmi. Un monitor *raggruppa procedure*, variabili e strutture dati. I processi possono chiamare le procedure di un monitor ma non accedere alla sua struttura interna.

- **Solo un processo può essere attivo in un monitor in un dato momento** - mutua esclusione
- **Il compilatore gestisce la mutua esclusione dei monitor** - minore probabilità di errori lato umano

I monitor utilizzano *variabili condizionali* e due operazioni su di esse: `wait` e `signal`, che *non accumulano segnali* (se `signal` viene inviato prima che il thread è in `wait` allora il segnale viene perso)

Sincronizzazione e barriere

Le **barriere** sono utilizzate per sincronizzare i processi in fasi diverse. Quando un processo raggiunge una barriera, attende finché tutti gli altri processi la raggiungono (`pthread_join`)

Read-Copy-Update

Problema dell'**inversione di priorità**:

- Un thread di alta priorità aspetta una risorsa bloccata da un thread di bassa priorità;
- Un thread di media priorità, che non ha nulla a che fare con la risorsa, impedisce al thread di bassa priorità di completare il suo lavoro.

Risultato: Il thread di alta priorità non riesce a lavorare anche se teoricamente dovrebbe avere la precedenza su tutti.

Soluzione: L'obiettivo del Read-Copy-Update è di accedere in modo concorrente senza lock, cercando di *evitare l'incosistenza dei dati*. L'idea è di aggiornare strutture dati consentendo letture simulate senza incappare in versioni inconsistenti dei dati. I lettori vedono o la versione vecchia o quella nuova, mai un mix delle due. E' diffuso nel kernel dei sistemi operativi.

Scheduling

Per evitare che processi/thread competano contemporaneamente per la CPU, si introduce lo **scheduler**, che decide quale processo/thread eseguire successivamente seguendo un **algoritmo di scheduling**.

I processi si possono dividere in:

- **CPU-bound:** Burst di CPU lunghi, attese di I/O infrequenti;
- **I/O-bound:** Burst di CPU brevi, attese di I/O frequenti.

Lo scheduling varia in base al contesto, ed opera in determinate situazioni:

- *Creazione di un nuovo processo*
- *Uscita di un processo*
- *Blocco del processo*
- *Interrupt I/O*

A loro volta, gli algoritmi di scheduling si suddividono in:

- **Non Preemptive** (senza prelazione) - Il processo viene eseguito fino al blocco o al rilascio volontario;
- **Preemptive** (con prelazione) - Il processo viene eseguito per un tempo fissato per poi essere sospeso

Ambienti di scheduling

Batch:

- Ideale per attività aziendali periodiche;
- Accetta algoritmi senza prelazione;
- Priorità a prestazioni efficienti.

Obiettivi:

- Massimizzare il **Throughput** - numero di job completati in un tempo fissato;
- Minimizzare il **tempo di Turnaround** - tempo dall'inizio alla fine di un processo;

- **Mantenere la CPU costantemente attiva.**

Interattivo:

- *Prelazione fondamentale;*
- Previene la monopolizzazione della CPU;
- Adatto per server ed utenti multipli.

Obiettivi:

- *Tempo di risposta* rapido alle richieste degli utenti;
- *Adeguatezza* per soddisfare le aspettative dell'utente in termini di tempi di risposta.

Real-Time:

- I processi spesso si bloccano velocemente sapendo di non poter eseguire a lungo;
- Prelazione non sempre necessaria;
- Eseguono programmi per specifiche applicazioni.

Obiettivi:

- *Rispetto delle scadenze* per completare i job nei tempi previsti;
- *Prevedibilità* per assicurarsi che il funzionamento sia costante per evitare degradi di qualità.

In generale, **tutti i sistemi** devono garantire:

- *Equità* - equa condivisione della CPU a tutti i processi;
- *Imposizione della policy* - dichiarata;
- *Bilanciamento* - mantenere attivi tutti i componenti del sistema

Scheduling nei Sistemi Batch

First-Come-First-Served(FCFS):

- Algoritmo di scheduling *senza prelazione*. I processi vengono assegnati alla CPU *nell'ordine di arrivo*. Si tratta di una singola coda di processi **ready**. Il primo job viene eseguito senza interruzioni, mentre quelli bloccati tornano in fondo alla coda.
- *Vantaggi*: Facile da capire e da programmare, equo in base all'ordine di arrivo.
- *Svantaggi*: Prestazioni non ottimali, si tramuta in tempi di attesa molto lunghi per processi I/O-bound se presenti anche processi CPU-bound.

Shortest Job First(SJF):

- Algoritmo di scheduling *senza prelazione*. Richiede che i tempi di esecuzione siano noti in anticipo, ed esegue per primo il job più breve.
 - *Vantaggi*: Ottimale per minimizzare il tempo di TurnAround medio quando i job sono tutti disponibili contemporaneamente.
 - *Svantaggi*: Non ottimale se i job arrivano in tempi diversi.
- Shortest Remaining Time Next (SRTN):**
- Variante di SJF *con prelazione*. Seleziona sempre il processo più corto per completare, ma ad ogni nuovo job confronta il suo tempo di esecuzione con quello del job attivo, e se più corto cambia contesto.
 - *Vantaggi*: Assicura che i job brevi ricevano un servizio rapido.
 - *Svantaggi*: Job più lunghi possono essere "rinviati" spesso.

Scheduling nei Sistemi Interattivi

Round-Robin Scheduling (RR):

- Ogni processo riceve un "*quanto*" di tempo per l'esecuzione. Se non ha terminato al termine del quanto, torna nella lista come `ready`, mentre se termina prima del quanto allora la CPU è oggetto di prelazione per un altro processo. Per implementarlo basta mantenere una lista dei processi eseguibili, e quando un processo esaurisce il quanto viene soistato alla fine della lista.
- La scelta della durata del quanto influisce sull'efficienza, quindi è opportuno scegliere un quanto ragionevole per bilanciare efficienza e reattività.

Priority Scheduling:

- Ogni processo ha una priorità assegnata, e la CPU esegue il processo con la priorità più alta tra quelli pronti. La priorità di un processo attualmente in esecuzione può diminuire col tempo, e se scende sotto quella del processo successivo, avviene un*cambio*.
- *Priorità statica vs dinamica*:
 - *Statica*: gerarchie assolute o basate sui costi nel data center;
 - *Dinamica*: gerarchie basate sull'utilizzo della CPU o sul comportamento I/O.
- Avviene un *raggruppamemto in classi* di priorità: 4 priorità, finché ci sono processi in priorità 4, si utilizza RR, poi si scende.

Shortest Process Next + Aging (SPN+):

- La sfida è identificare quale tra i processi eseguibili sia effettivamente il più breve.
- *Aging* - Stima basata sul comportamento passato:
 - Stima del tempo per un comando T_0 ;
 - Stima aggiornata dopo nuova esecuzione T_1 diventa $\alpha T_0 + (1 - \alpha)T_1$.

- La scelta di α determina il peso delle esecuzioni precedenti nella nuova stima.

Guaranteed Scheduling:

- Il *concetto principale* è fare promesse concrete sugli standard di prestazione e rispettarle. Se ci sono n processi, ciascuno ottiene $\sim \frac{1}{n}$ della potenza della CPU.
- Il sistema tiene traccia di quanta CPU ha ricevuto ogni processo dal momento della sua creazione, e calcola quanto tempo di CPU ogni processo dovrebbe avere. Poi valuta il rapporto tra tempo consumato e dovuto, ed esegue il processo con il rapporto più basso finché non supera il suo concorrente più vicino.

Lottery Scheduling:

- Assegnazione di biglietti della lotteria ai processi per la risorsa del sistema (es. tempo CPU), ed estrazione casuale di un biglietto per decidere quale processo ottiene la risorsa.
- Avviene una *distribuzione delle probabilità* per la quale vengono assegnati biglietti extra per i processi più importanti, che gli garantiscono maggiore probabilità di vincere.
- Il sistema è reattivo e permette la *cooperazione tra processi* nello scambiarsi biglietti (utile per processi cooperanti).

Fair-Share Scheduling:

- Premessa: ogni processo è oggetto di scheduling individualmente.
- L'approccio fair-share considera le proprietà di ogni processo prima di sceglierlo. Ogni utente riceve una frazione predefinita di CPU, e lo scheduler si assicura che ogni utente riceva la sua frazione, indipendentemente dal numero di processi posseduti.

Scheduling nei Sistemi Real-Time

Viene usato nei SO in applicazioni in cui il tempo di risposta è fondamentale, e ritardi o mancati tempi di risposte possono avere gravi implicazioni. Si suddividono in:

- *Hard Real-Time*: Scadenze assolute da rispettare;
- *Soft Real-Time*: Leggera tollerabilità.

I processi sono prevedibili, brevi e noti in anticipo, e possono essere *periodici* o *non periodici*.

La *condizione di schedulabilità* è regolata dalla capacità della CPU di gestire la somma totale del tempo richiesto dai processi.

Se ci sono m eventi periodici, l'evento i avviene con un periodo P_i e richiede C_i secondi di tempo della CPU per gestire ogni evento, allora il carico può essere

gestito solo se::

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

Scheduling di thread

Lo scheduling differisce in base al tipo di thread, se sono a livello **utente** o **kernel**.

- **Thread a livello utente**: il kernel vede solo il processo ed ignora l'esistenza dei singoli thread che lo compongono (quindi il kernel gestisce solo i processi e non i thread individuali). Esiste uno **scheduler interno** ad ogni processo che decide quale thread eseguire, senza interruzioni del clock del sistema. Poiché il kernel assegna la CPU all'intero processo e non ai singoli thread, un thread a livello utente può consumare tutto il quanto di tempo assegnato al processo. Lo scambio tra thread avviene inoltre molto rapidamente con poche istruzioni, poiché non si deve passare per ogni thread alla modalità kernel.
- **Thread a livello kernel**: il kernel seleziona il thread specifico per l'esecuzione. Se eccede il quanto, allora viene sospeso. In questo caso il passaggio tra thread richiede un **cambio di contesto** e quindi comporta un passaggio da modalità utente a kernel, ergo tempi di maggiori di esecuzione. Se un thread kernel richiede I/O non sospende stavolta tutto il processo, ma solo il thread in questione.